Returning Status

Values and control flow

Gregory S. DeLozier, PhD

Breaking Changes

```
value, status = evaluate(ast, environment)
```

Every evaluator result is now a pair.

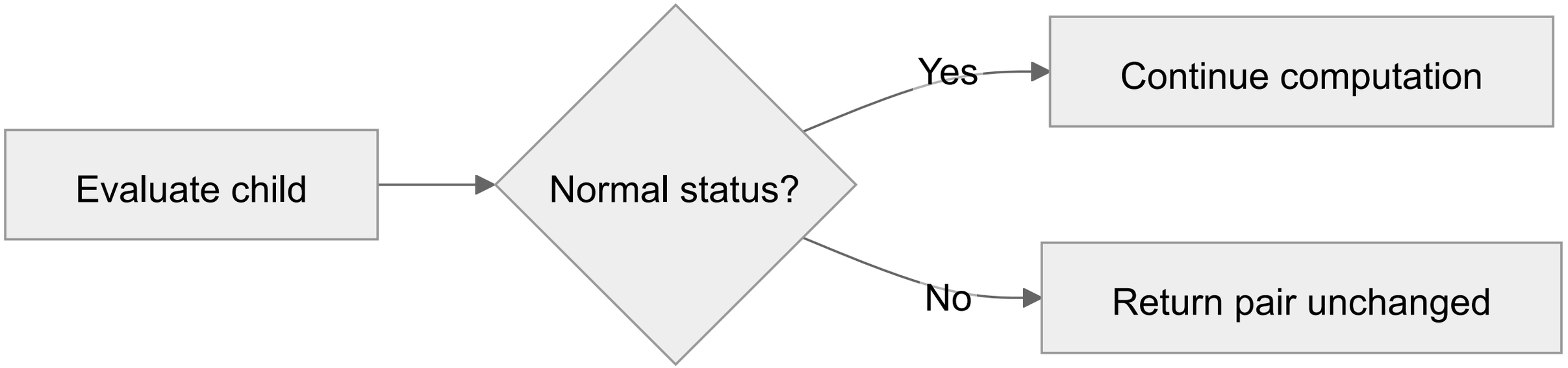
`exit` and `assert` become keywords.

Value Is Not Status

Evaluation	Value	Status
`2 + 3`	`5`	`None`
`2 > 3`	`False`	`None`
`print(5)`	`None`	`None`
`exit(7)`	`7`	`"exit"`

False is a normal result.

The Caller Rule



Check status before using the value.

Two Operands, Two Checks

```
left, status = evaluate(ast["left"], environment)
if status is not None:
    return left, status
right, status = evaluate(ast["right"], environment)
if status is not None:
    return right, status
return left + right, None
```

Requesting Exit

Source	Returned pair
`exit()`	`(0, "exit")`
`exit(2 + 3)`	`(5, "exit")`

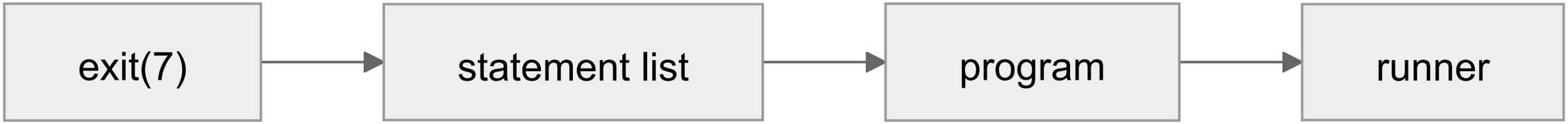
Parentheses required. Integer argument optional. A statement, not an expression: cannot nest inside `1 + exit(7)` or `a[exit(7)]`.

Where Does Execution Stop?

```
x = 3;
exit(7);
x = 99;
print(x);
```

No reassignment to 99. No print. The first assignment already ran.

Exit Returns Outward



Every arrow carries `(7, "exit")`.

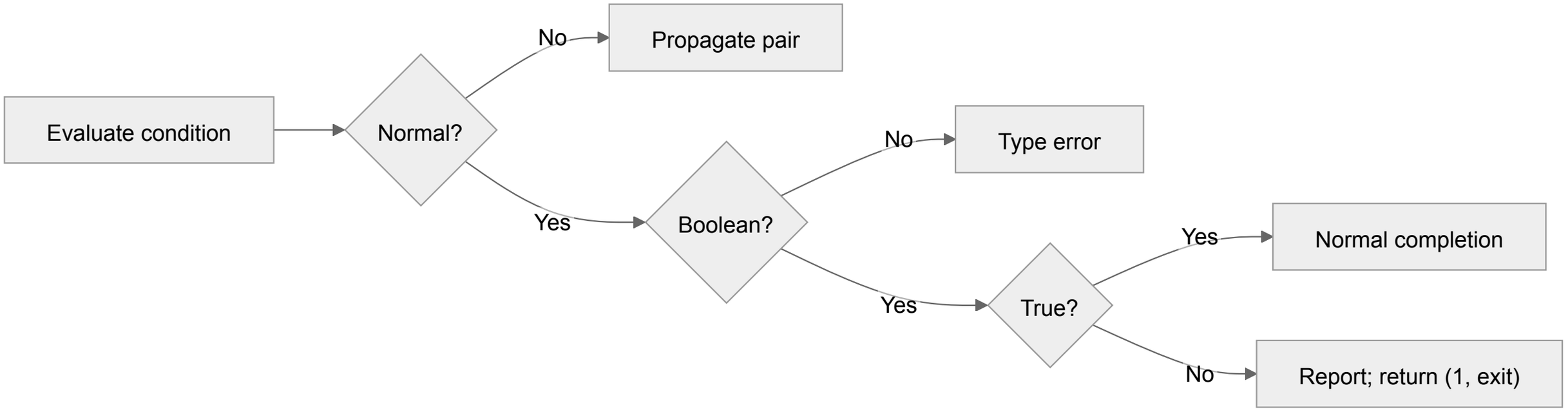
Assertions State Expectations

```
answer = 41;
assert answer == 42, "Expected 42";
print("This must not run");
```

```
Assertion failed: Expected 42
```

Report here. Return `(1, "exit")`.

Assertion Workflow



The Runner Handles Exit

```
value, status = evaluate(ast, {})
if status == "exit":
    sys.exit(value)
if status is not None:
    raise ValueError(f"Unhandled status: {status}")
```

Evaluation is testable without terminating the test process.

Skipped Nodes Produce No Status

Expression	Result
`false and (1 / 0)`	`(False, None)`; division never runs
`true or (1 / 0)`	`(True, None)`; division never runs
`true and (1 / 0)`	`ZeroDivisionError`; division does run
`false or (1 / 0)`	`ZeroDivisionError`; division does run

Stopping Is Not Undoing

```
result = input() + (1/0)      input is consumed, then crash
result = (1/0) + input()      crash first, input is skipped
```

Both raise **`ZeroDivisionError`**.

Test the Pair and the Effects

```
result, env = evaluate_source("x=3;exit(7);x=99")
assert result == (7, "exit")
assert env["x"] == 3
```

Exit exactly once. Later work never runs.

Questions for Thought

Why check status before storing an assignment?

Which structure should handle `break`?

What would recovery from assertion failure require?